

NaN Root Cause Analysis and Fix Plan for quadrature_analyzer_fixed.py

Executive Summary

All 7 demo cases produce NaN when running `python api/quadrature_analyzer_fixed.py` as `__main__`. The root cause is a **sys.path issue**: Python adds the script's directory (`api/`) to `sys.path[0]`, causing all internal imports like `from legendre import LegendreQuadrature` to fail silently (caught by the broad `try/except` in `execute_quadrature`).

When imported as a module (e.g., `from api.quadrature_analyzer_fixed import QuadratureAnalyzer`), the code works correctly because the project root is on `sys.path`.

Root Cause: sys.path When Running as main

Mechanism

```
python api/quadrature_analyzer_fixed.py    # FAILS - all NaN
```

When Python runs a script, it adds the **script's directory** to `sys.path[0]`:

- `sys.path[0] = C:\...\OrthoPolyBasis0k-main\api` (WRONG)
- Imports like `from legendre import LegendreQuadrature` look in `api/legendre/` which doesn't exist

```
# In execute_quadrature():
try:
    value = self._integrate_legendre(func, ...) # ImportError caught here!
except Exception as e:                        # Broad except catches it
    return QuadratureResult(                  # Returns NaN silently
        value=float("nan"), ...
        message="Quadrature computation failed: " + str(e),
    )
```

Verification

```
# Running from api/ directory context:
sys.path[0] = 'C:\...\OrthoPolyBasis0k-main\api' # WRONG - no legendre here!
from legendre import LegendreQuadrature          # ImportError!
```

Fix Strategy

FIX 1: Add project root to sys.path in main block (Primary fix)

Add at the top of `if __name__ == "__main__":`:

```
import os, sys
# Ensure project root is on sys.path so that 'from legendre import ...' works
_project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if _project_root not in sys.path:
    sys.path.insert(0, _project_root)
```

FIX 2: Narrow the exception handling in execute_quadrature (Defensive fix)

The current broad `except Exception` silently catches import errors and returns NaN. Split into specific handlers:

Current code:

```
try:
    value = self._integrate_legendre(func, ...)
except ImportError as e:
    return QuadratureResult(value=float("nan"), ..., message="Cannot import...")
except Exception as e:
    return QuadratureResult(value=float("nan"), ..., message="Quadrature failed...")
```

Problem: The broad `except Exception` catches everything including unexpected errors, making debugging impossible.

Fix: Add a print/traceback for the ImportError case to surface import issues early:

```
except ImportError as e:
    # Print traceback so user can see what went wrong
    import traceback
    traceback.print_exc()
    return QuadratureResult(value=float("nan"), ..., message="Import error...")
```

Secondary Issues Found During Analysis

ISSUE A: Case 4 ($\log(x) \cdot \exp(-x)$) - Does not converge within $\text{tol}=1\text{e-}12$

Symptom: converged=False with $n=128$, $\text{err}=1.773\text{e-}03$ **Root cause:** $\log(0)$ singularity at $x=0$ is inherent to the integrand. Gauss-Laguerre converges algebraically ($O(n^{-1/2})$) for log-singular integrands as noted in the degree criteria text itself.

Fix options:

1. Increase suggested_max_n from 128 to higher values when log singularity detected
2. Use Generalized Laguerre with $\alpha > 0$ (already suggested in criteria)
3. Accept that this case won't converge at $\text{tol}=1\text{e-}12$ and document it

Recommendation: This is expected behavior, not a bug. The analyzer correctly identifies the issue in its degree criteria output. No code change needed, but the demo should acknowledge this limitation.

ISSUE B: Case 7 ($1/(1+x^4)$) - Wrong family recommendation

Symptom: Recommends Hermite for algebraic decay on infinite interval **Root cause:** `_probe_decay` returns ("none", 0.0) because $1/(1+x^4)$ doesn't match any of the recognized patterns (Gaussian, exponential, log). The function has algebraic decay $O(x^{-4})$ but this is not detected.

Fix in `_infer_interval / _probe_decay`:

```
# After checking for Gaussian and exponential, check for rational/algebraic decay:
# Pattern: 1/(x^n + ...) or x^(-n) => algebraic decay on infinite interval
try:
    numer, den = expr.as_numer_denom()
    if den.has(v):
        # Check degree of denominator vs numerator
        from sympy import degree as sym_degree
        deg_den = den.degree(v) if hasattr(den, 'degree') else None
        deg_num = numer.degree(v) if hasattr(numer, 'degree') and numer != 1 else 0
```

```
        if deg_den is not None and deg_den > deg_num:
            return ("algebraic", float(deg_den - deg_num))
    except Exception:
        pass
```

Fix in `_recommend_family`: When `decay_type == "algebraic"` on infinite interval, recommend Legendre with a finite truncation instead of Hermite.

ISSUE C: Convergence check uses `err < tol` which is too strict for near-zero integrals

Symptom: Case 1 ($\exp(-x^2)\sin(x)$) has true value ~ 0 but reports `rel_err=1.302e+00` **Root cause:** When the integral value is very small ($\sim 1e-17$), even tiny absolute errors produce huge relative errors.

Current code:

```
converged = (err < tol) or (rel_err < tol)
```

This actually works correctly for Case 1 because `err=1.538e-17 < 1e-12` is True, so it converges via the absolute error check. However, the `rel_err` display is misleading.

Recommendation: No code change needed - the convergence logic is correct (absolute OR relative). The DEBUG output could be improved to show which criterion passed.

Implementation Order

1. **FIX 1:** Add `sys.path` fix in `__main__` block — resolves ALL NaN issues
2. **FIX 2:** Improve exception handling for better diagnostics
3. **ISSUE A:** Document as expected behavior (no code change)
4. **ISSUE B:** Improve algebraic decay detection and family recommendation
5. **ISSUE C:** No change needed

Files Modified (Only This One)

- `api/quadrature_analyzer_fixed.py` — the only file that needs modification

Files NOT Modified (Read-Only Constraint)

- `chebyshev/*` — never modified
- `hermite/*` — never modified
- `laguerre/*` — never modified
- `legendre/*` — never modified